

AI BLOGNEST API

RESTful Backend with AI-Powered Blog
Generation and Summarization BSc
Computer Science Project Report

Project Team

Team Leader:

R. Kalidass

Team Member: S. Jageer

Team Member: M. Harish

Team Member: R. Vinothkumar

1. INTRODUCTION

AI BlogNest API is a RESTful backend application developed using Node.js, Express.js, MongoDB, Mongoose, JWT authentication, bcrypt, and Gemini AI.

The system provides user registration and login, secure authentication, complete blog CRUD operations, AI blog content generation, and AI summarization. The project follows MVC architecture to keep the backend modular and maintainable.

Main Features

- User registration and login
- JWT-protected routes
- Blog creation, reading, updating, and deletion
- AI blog generation
- AI summarization
- MongoDB data storage
- MVC architecture

2. OBJECTIVES

The main objective is to develop a modern and secure backend that combines REST API development with generative AI.

Objectives

- Develop a RESTful backend using Node.js and Express.js.
- Provide user registration and login.
- Implement JWT authentication.
- Secure passwords using bcrypt.
- Provide complete blog CRUD operations.
- Integrate Gemini AI for blog generation.
- Provide AI-powered summarization.
- Store application data in MongoDB.
- Follow MVC architecture.
- Provide APIs that can connect with frontend applications.

Expected Outcome

The completed system provides a reusable backend through which authenticated users can manage blogs and use AI assistance. Its modular design supports future expansion.

3. TECHNOLOGIES USED

Node.js: Backend runtime environment.

Express.js: Framework used to create REST API routes and handle HTTP requests.

MongoDB: NoSQL database used to store users and blog information.

Mongoose: Provides schemas and models for MongoDB interaction.

JWT: Used for secure user authentication and protected routes.

bcrypt: Used to hash user passwords before database storage.

Gemini AI: Used for AI blog generation and summarization.

JavaScript: Main programming language.

Thunder Client/Postman: Used to test API endpoints.

dotenv: Used to manage environment variables and sensitive configuration.

4. SYSTEM ARCHITECTURE

AI BlogNest API follows the MVC (Model-View-Controller) architecture.

Model Layer: Manages database structures using Mongoose and MongoDB.

Controller Layer: Contains application logic, validation, database operations, and AI service calls.

Route Layer: Defines API endpoints and connects requests to controllers.

Middleware: Verifies JWT tokens before protected operations.

AI Service: Communicates with Gemini for content generation and summarization.

Database: MongoDB stores user accounts, blog posts, and metadata.

Request Flow: Client → Express Route → Middleware → Controller → Model/AI Service → MongoDB or Gemini → JSON Response.

5. USER AUTHENTICATION MODULE

The authentication module provides secure account management.

Registration: The user submits required information such as name, email, and password. The password is hashed using bcrypt before storage.

Login: The system verifies the email and password. Valid credentials result in a JWT token.

JWT Authentication: The token is sent with protected requests and verified by authentication middleware.

Profile: Authenticated users can retrieve their profile information.

Authentication APIs

POST /api/auth/register

POST /api/auth/login

GET /api/auth/profile

This module helps prevent unauthorized access and avoids storing passwords as plain text.

6. BLOG MANAGEMENT MODULE

The Blog Management Module provides complete CRUD functionality.

Create: Authenticated users can create a blog using a title, content, and required metadata.

Read: Users can retrieve all blogs or a particular blog using its ID.

Update: Existing blog information can be modified through the update endpoint.

Delete: An existing blog can be removed using its unique ID.

Blog APIs

POST /api/blogs

GET /api/blogs

GET /api/blogs/:id

PUT /api/blogs/:id

DELETE /api/blogs/:id

Workflow: Client request → Authentication → Controller → Mongoose → MongoDB → JSON response.

7. AI CONTENT GENERATION

AI-powered content generation is a major feature of AI BlogNest.

Endpoint: POST /api/ai/generate-blog

The user submits a topic or prompt. The backend validates the request and sends an appropriate prompt to Gemini AI. The generated content is returned to the client as an API response.

Process

User Prompt → Express API → AI Controller/Service → Gemini AI → Generated Content → API Response

Benefits

- Helps create initial blog drafts quickly.
- Reduces manual writing effort.
- Generates structured content from topics.
- Provides AI-assisted writing support.

The Gemini API key and model configuration should be stored securely in environment variables.

8. AI SUMMARIZATION AND DATABASE

AI BlogNest also provides AI-powered summarization.

Endpoint: POST /api/ai/summarize

The user submits existing blog content. The backend sends the content to Gemini AI, which generates a shorter summary and returns it through the API.

MongoDB Database

MongoDB is the primary database, while Mongoose provides schemas and models.

Typical Data

- User information
- Hashed passwords
- Blog titles
- Blog content
- Author information
- Created and updated timestamps

The MongoDB connection is configured using the MONGO_URI environment variable.

9. API TESTING AND SECURITY

The APIs can be tested using Thunder Client or Postman.

Testing Areas

- Registration and login
- JWT authentication
- Profile retrieval
- Blog creation and listing
- Blog updating and deletion
- AI blog generation
- AI summarization

Security Features

- Password hashing with bcrypt
- JWT-based authentication
- Protected routes
- Environment variables for secrets
- Secure database configuration
- Input validation and error handling
- API keys kept outside source code

Environment Variables

PORT

MONGO_URI

JWT_SECRET

GEMINI_API_KEY

GEMINI_MODEL

10. CONCLUSION AND FUTURE ENHANCEMENT

Conclusion

AI BlogNest API successfully provides a RESTful backend for an AI-powered blogging platform. It combines Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Gemini AI for authentication, blog management, content generation, and summarization.

The MVC architecture provides clear separation between components. JWT and bcrypt provide important security mechanisms, while MongoDB provides flexible data storage. Gemini integration adds AI-assisted functionality to the blogging workflow.

Future Enhancements

- Blog categories and tags
- Image upload functionality
- Comments and likes
- Admin and author roles
- Search and filtering
- Pagination
- AI grammar correction
- AI SEO recommendations
- Social media sharing
- Cloud deployment
- Analytics dashboard

Final Statement

The project demonstrates how modern backend technologies and generative AI can be combined to create a practical, modular, and extensible web application backend.